

CSC490 Assignment A2

Data Processing Pipelines and Infrastructure as Code

Ananya Jain (1007706573), **Aviral Bhardwaj** (1008239407)
Ansh Agrawal (1007413103), **Mann Thakkar** (1007647744)

Due: February 20, 2026

1 Aspirational Datasets for Metabolic Routing

These specialized datasets are designed to train, evaluate, and analyze the temporal and dynamic aspects of metabolic routing in mixture-of-experts architectures. Unlike standard MoEs, metabolic routing depends on historical load, fatigue state, and recovery capacity. Therefore, these datasets prioritize sequence-level trajectories, burstiness, and stateful evaluation over static token-level assignments.

1.1 Routing Trajectory Dataset (Ground-Truth)

Purpose:

- Provide ground-truth supervision for routing that explicitly accounts for metabolic constraints (fatigue and recovery rates).
- Train the router to learn an optimal sequence-level routing path rather than greedy, static token-level assignments.

Schema:

- `input_ids`: `List[int]` — Tokenized input sequence.
- `attention_mask`: `List[int]` — Attention mask for valid tokens.
- `oracle_expert_path`: `List[int]` — The globally optimal sequence of expert assignments computed via dynamic programming (e.g., Viterbi) given a strict metabolic budget.
- `alternate_experts`: `List[List[int]]` — Suboptimal but “fresh” alternatives for each token, ranked by acceptability.
- `loss_tolerance`: `List[float]` — The local acceptable increase in loss if forced to route to a fallback expert due to fatigue.

1.2 Cross Domain Dataset

Purpose:

- Train the router’s understanding of domain “burstiness” and its ability to manage expert recovery windows.
- Understand how experts handle heterogeneous tasks when domains are heavily concentrated (forcing saturation) versus sparsely interleaved (allowing recovery).

Schema:

- `input_ids`: `List[int]` — Tokenized input sequence.

- `task_id: str` — Task category (e.g., “code”, “math”, “narrative”).
- `burst_intensity: float` — Measure of how concentrated the task is within the local context timeframe.
- `recovery_window: int` — The number of tokens since this specific domain was last active (measures expert freshness).

1.3 Metabolic Stress-Test Suite

Purpose:

- Create controlled, synthetic scenarios to unit-test the router’s fatigue and recovery hyper-parameters (β_{cost} , $\gamma_{recovery}$).
- Validate that the router successfully initiates fallback routing when saturation occurs, and returns to primary experts upon recovery.

Schema:

- `input_ids: List[int]` — Synthetic tokenized sequence designed to stress the system.
- `stress_profile: str` — Profile type (e.g., “sustained_monoculture_load”, “rapid_alteration”, “starvation_and_surge”).
- `primary_target_expert: int` — The expert that naturally specializes in these tokens.
- `expected_fallback_expert: int` — The expert that should take over at the exact token index where the primary reaches metabolic exhaustion.
- `exhaustion_token_idx: int` — The theoretical index where metabolic saturation should force a route change based on the system’s math.

1.4 Temporal Distributional Shift Ecosystem (Evaluation)

Purpose:

- Assess whether the router gracefully degrades and re-allocates capacity during massive, unanticipated shifts in topic distribution.
- Evaluate how quickly the system forgets a previous domain and recharges its experts for the new dominant domain.

Schema:

- `input_ids: List[int]` — Tokenized data streams representing chronological inputs.
- `timestamp / step_index: int` — Temporal tracking index.
- `shift_event_id: str` — Marker for when a known macro-shift occurs (e.g., transitioning abruptly from a math paper corpus into a Python codebase).
- `domain_drift_score: float` — Quantitative measure of the distance from the moving average of the domain distribution.

2 Available Datasets

Dataset	Description	Relevance
WikiText-103 (+ small Pile / Open-WebText subset)	Medium-scale English language modeling corpora (Wikipedia-like + web text) commonly used for GPT-Neo and similar LMs.	LM POC on GPT-Neo: Train/fine-tune GPT-Neo 125M/1.3B (dense vs. MoE) and test whether the metabolic router is stable and competitive in perplexity. Later reuse for larger LLMs to compare routers at fixed FLOPs.
C4-Small (Calibration Slice)	Small, fixed slice of the C4 corpus (e.g., 100k–500k tokens) used only to log routing statistics.	Utilization / fatigue calibration: Estimate expert utilization, fatigue trajectories, and drive offline what-if analyses (apoptosis / mitosis / fusion). Also enables comparison to routing-statistics-based methods (M-SMoE / DERN-style).
GLUE	Standard NLU benchmark (MNLI, QQP, SST-2, QNLI, etc.) for sentence- and pair-level classification.	NLU under different routers: Fine-tune GPT-Neo-based classifiers (dense vs. various MoE routers) and later larger LLMs. Compare accuracy and calibration across routers at matched compute; inspect whether metabolic routing changes expert specialization by task.
SuperGLUE (optional)	Harder NLU benchmark (CB, RTE, WSC, COPA, etc.) focused on challenging reasoning and coreference.	Challenging NLU stress-test: After GLUE POC, use SuperGLUE to test whether the router improves or maintains performance for harder reasoning compared with top- k and DynMoE-style routers.
HellaSwag , PIQA , ARC-E / ARC-C	Commonsense and science multiple-choice benchmarks widely used for LLM evaluation.	Commonsense / science reasoning: Evaluate dense vs. MoE LMs (with different routers) on these datasets to determine if metabolic birth–death–fusion preserves rare-but-important reasoning experts better than standard routing.
MMLU	Massive Multitask Language Understanding: 57 academic subjects across diverse domains.	Large-scale downstream for scaled LLMs: Once the router is integrated into larger MoE LLMs (Mixtral-/Qwen-MoE-class), compare MMLU accuracy vs. Switch-style top- k , DynMoE, and SMEAR at fixed or reduced FLOPs.

Dataset	Description	Relevance
Alpaca / FLAN-like Instruction Mixes	Instruction-following datasets (e.g., Alpaca-style synthetic instructions or FLAN-style mixtures) used to align base LMs.	Instruction diversity and specialization: Fine-tune GPT-Neo and later larger LLMs to study whether the metabolic router yields experts specialized by instruction type (reasoning vs. classification vs. coding prompts), and compare helpfulness/robustness across routers.
CodeParrot-Small / GitHub Python Subset	Medium-scale Python code corpus (functions, scripts, small projects) suitable for fine-tuning small and medium LMs.	Traffic source for code experts: Train GPT-Neo on mixed text+code to generate diverse expert usage. Later reuse for larger LLMs to analyze which experts specialize on code vs. natural language and how fatigue/apoptosis shape that distribution.
HumanEval	Small benchmark of hand-written Python programming problems with unit tests; standard pass@k metric for code generation.	Code generation evaluation: On GPT-Neo POC and larger LLMs, compare pass@1 / pass@10 across routers (dense, top- <i>k</i> , DynMoE, SMEAR, metabolic). Check whether metabolic routing harms or improves difficult long-horizon coding tasks.
MBPP	1k mostly basic Python programming problems with reference solutions and tests.	Stable statistics for code routing: Complement HumanEval with more problems to analyze expert usage and specialization (e.g., experts for loops vs. strings vs. math). Compare routing-induced specialization patterns across routers.
SVAMP (+ related MAWPS-style word problems)	Small math word-problem datasets requiring simple algebraic reasoning and translation from text to equations.	Lightweight math POC: Early testing of math reasoning under different routers on GPT-Neo, before moving to larger math suites. Good for ablations on fatigue dynamics and expert turnover with minimal compute.
GSM8K	Grade-school math word problems with step-by-step solutions; popular for chain-of-thought math reasoning.	Math reasoning and CoT: Fine-tune GPT-Neo and larger LLMs with or without chain-of-thought. Compare accuracy and solution length across routers, and examine whether metabolic mitosis/apoptosis yields stable math-specialized experts.
MATH	Competition-style math problems covering algebra, geometry, number theory, etc., with difficulty labels.	High-difficulty math at scale: On larger LLMs, test whether the router maintains math performance compared to strong baselines (dense, top- <i>k</i> , DynMoE, SMEAR), and whether lineage-aware fusion/apoptosis damages rare but complex subdomains.

2.1 Summary of Datasets and Evaluation Coverage

Although the aspirational datasets could not be obtained, we have curated the above-mentioned dataset suite, which is currently used in industry to train and evaluate router mechanisms. This suite will be utilized across stages (where feasible) to compare our router mechanism and existing state-of-the-art (SOTA) and industry-standard routers. While not all collected datasets may ultimately be used, we have assembled a comprehensive set to ensure broad coverage for router training.

Below are the datasets we currently support and will use as the core training resources for our study:

Training and Adaptation Data

- **General Text (Steady-State Baseline):** WikiText-103 and Pile-like subsets establish and evaluate the router’s *steady-state metabolic equilibrium* under balanced heterogeneous loads.
- **Instruction Tuning (Context Switching):** Alpaca/FLAN-style instruction mixtures evaluate rapid domain alternation and expert recovery ($\gamma_{recovery}$) across disjoint tasks.
- **Code Pretraining (Sustained Saturation):** Continuous CodeParrot (GitHub Python) streams apply prolonged pressure to syntax/logic experts to induce fatigue and observe fallback routing.
- **Mathematical Reasoning (Chain-of-Thought):** SVAMP, GSM8K, and MATH assess whether experts can sustain long sequential reasoning chains.

Benchmarks (Tasks and Baselines)

- **Baselines for Comparison:** (1) *Standard Top-K MoE* (e.g., Switch Transformer without fatigue), (2) *Dense Model Baseline* with equivalent active parameters, and (3) *Random Routing* as a lower bound.
- **Language Modeling:** WikiText-103 and OpenWebText measure whether metabolic constraints affect base generation quality.
- **Domain-Specific Reasoning:** GSM8K (math) and HumanEval (code) evaluate robustness under bursty, long-context workloads and potential expert exhaustion.

Router-Specific Evaluation Metrics

- **Perplexity (PPL):** Standard predictive quality metric; tracked over long sequences to quantify fatigue-induced degradation.
- **Gini Coefficient:** Measures inequality in expert assignment; lower values indicate better load balance.
- **Expert Utilization Entropy:** Quantifies distribution uniformity across experts; higher entropy indicates healthier routing.
- **Effective Experts (N_{eff}):** Estimates the number of experts meaningfully contributing during inference.
- **Temporal Expert Usage:** Token-level routing trace over time to verify expert rotation under fatigue and recovery.
- **Fatigue Saturation Rate (F_{sat}):** Fraction of steps where an expert reaches maximum metabolic capacity ($F_i \approx 1.0$).

3 Part Three: Data-Processing Pipelines

3.1 Data Ingestion Pipeline

The data ingestion pipeline is responsible for retrieving raw datasets from external sources (Hugging-Face Hub), validating their structure, and storing them immutably in the S3-based data lake. The pipeline is implemented as a SageMaker Processing job orchestrated via a local controller script.

3.1.1 Data Schemas

Data flows through the ingestion stage in two structured forms:

1. Raw Output (Per Split) — JSONL (Default Format). Each dataset split (train, validation, test) is written as a separate JSONL file (i.e `train.jsonl`)

Sample data input of `wikitext-2-raw-v1`, which are stored in JSONL files:

```
{"text": "= Valkyria Chronicles III =\n"}
{"text": "Senj no Valkyria 3 : Unrecorded Chronicles ..."}

```

2. Provenance Record — `metadata.json` A metadata file is written alongside the splits to ensure reproducibility and auditability:

```
{
  "dataset_name": "KrisMinchev/wikitext-2-raw-v1",
  "processing_timestamp": "2025-02-19T14:30:00+00:00",
  "output_format": "jsonl",
  "splits": {
    "train": {"num_examples": 36718, "file_size": 12451200},
    "validation": {"num_examples": 3760, "file_size": 1280000},
    "test": {"num_examples": 4358, "file_size": 1490000}
  },
  "total_records": 44836,
  "total_bytes": 15221200
}
```

S3 Data Lake Layout Each run writes to a timestamped prefix to ensure immutability:

```
s3://tmoe-raw-data-dev-<suffix>/datasets/raw/<timestamp>/
  train.jsonl
  validation.jsonl
  test.jsonl
  metadata.json

```

Each execution writes to a new timestamped prefix, ensuring previous versions remain untouched. This design guarantees:

- Versioned dataset snapshots
- Safe rollback to previous dataset versions
- Fallback data ensures continuity during primary source (Hugging Face) downtime

3.1.2 Pipeline diagram

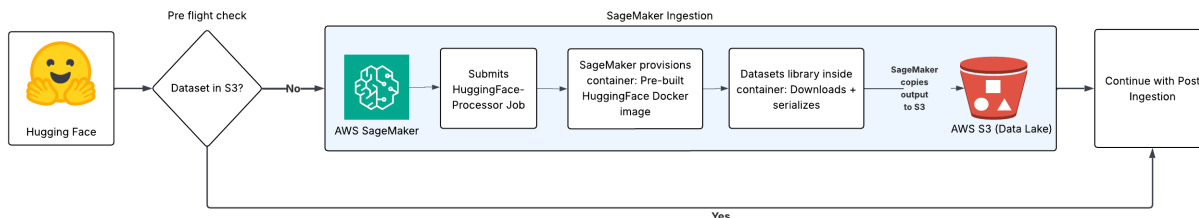


Figure 1: Data Ingestion Pipeline

3.1.3 When the Pipeline Runs and For Which Use Cases

The ingestion pipeline is triggered manually (or via CI/CD) rather than on a fixed schedule, since the source dataset is static. The data ingestion pipeline operates under the following scenarios:

Trigger	Use Case
Pre-flight check	Populate the S3 data lake (if it doesn't exist) before any training run
Dataset change	When training on a different HuggingFace dataset
Output format change	Re-ingest in Parquet instead of JSONL
Disaster Management	Re-ingest from source when struck with a disaster (i.e S3 bucket is deleted)

3.1.4 Code for Initial Pipeline Implementation

The ingestion pipeline is split into two components:

- **Orchestrator (local)** — Submits SageMaker Processing job
- **Worker (containerized)** — Performs dataset loading, validation, and writing

Orchestrator — `run_processing.py`

```

processor = HuggingFaceProcessor(
    role=config.sagemaker_role_arn,
    instance_type="ml.m5.xlarge",
    transformers_version="4.26",
    pytorch_version="2.0",
    py_version="py310",
    base_job_name="tmoe-data-ingestion",
)

processor.run(
    code="processing_script.py",
    source_dir="infra/data_ingestion/",
    outputs=[ProcessingOutput(
        source="/opt/ml/processing/output",
        destination=f"s3://{config.raw_data_bucket}/datasets/raw/{timestamp}/",
    )]
)
  
```

```

    ]],
    environment={
        "DATASET_NAME": "KrisMinchev/wikitext-2-raw-v1",
        "OUTPUT_FORMAT": "jsonl"
    },
    wait=True,
)

```

Worker — processing_script.py

```

def main():
    validate_environment() # disk space, network, writability

    dataset = load_huggingface_dataset(DATASET_NAME)

    for split_name, split_data in dataset.items():
        validate_split_data(split_name, split_data)
        write_split_to_disk(
            split_name,
            split_data,
            OUTPUT_FORMAT
        )

    write_metadata()
# SageMaker automatically uploads
# /opt/ml/processing/output S3

```

3.1.5 Next Steps

Several production-grade features are planned but not yet implemented:

Feature	Why It Matters
Scheduled triggers	Automate runs via EventBridge or CI cron jobs when new dataset versions are published
Schema validation (Great Expectations)	Enforce strict data contracts (length bounds, null rates, distributions)
Partitioned S3 layout	Partition by year/month/day for Athena-optimized querying
Multi-dataset support	Parameterize pipeline to ingest multiple datasets in <i>parallel</i>

This ingestion layer forms the immutable foundation of the data lake and ensures reproducibility, auditability, and safe experimentation for downstream model training and evaluation.

3.2 Training Compute Pipeline

The T-MoE pipeline supports two compute backends for model training: **AWS Batch** and **Modal** unified under a single entry point: `run_tmoe.py`. Both backends read dataset splits from the same S3 prefix and write experiment outputs (checkpoints, configuration, metrics) back to the same S3 bucket and online performance metrics tracking platform *Weights & Biases*.

S3 acts as the handshake layer between storage and compute. This decoupling ensures:

- Durable artifacts storage
- Compute backend interchangeability (AWS Batch $\leftrightarrow$ Modal)
- Reproducible experiment storage

3.2.1 Experiment Output Schema

At the end of every training run, `upload_experiment_dir()` (defined in `infra/s3client/s3_sync.py`) recursively syncs the *training output files* to directory with this path:

```
s3://tmoe-raw-data-dev-<suffix>/experiments/<run-name>/
```

S3 Structure After a Completed Run

```
experiments/<run-name>/
checkpoints/
  step_1000.pt
  step_2000.pt
  best_model.pt
config.yaml
wandb/
```

Directory Components

- **checkpoints/** Contains PyTorch state dictionaries (.pt files) managed by `CheckpointManager` (`src/training/checkpoint.py`). Maintains a rolling window of recent checkpoints while preserving `best_model.pt` (lowest training loss).
- **config.yaml** Frozen experiment hyperparameters (model size, learning rate, batch size, backend choice). This file is the **single source of truth** for reproducibility. Any experiment can be reproduced by passing this file back into `run_tmoe.py`.
- **wandb/** Local Weights & Biases metadata and artifact mirror. Real-time metrics (loss, perplexity, router load distribution, learning rate) stream to the W&B cloud during training, while this folder is synced to S3 for archival.

3.2.2 Pipeline Diagram

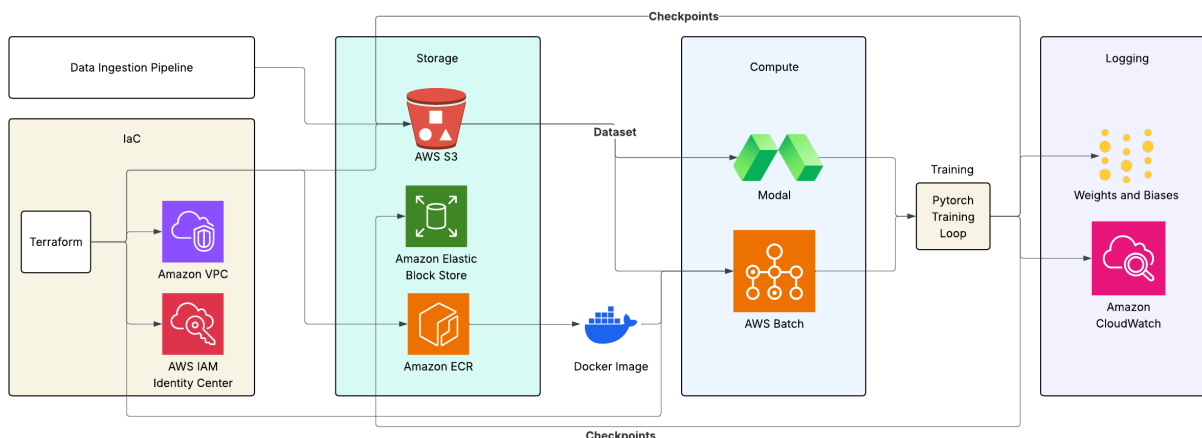


Figure 2: High-Level Pipeline Architecture

3.2.3 Pipeline Invocation and Use Cases

Both compute backends are invoked through the same entry point:

```
python run_tmoe.py --config <experiment_name> --backend [aws | modal]
```

The backend may also be defined in experiment’s `config.yaml` and overridden at the command line. We decided on a multi-environment setup for executing our training loops to accommodate these following conditions:

Condition	Backend	Reason
Long multi-hour production run	AWS Batch	Uses provisioned infrastructure; suitable for unattended runs; writes directly to production S3
Rapid iteration / debugging	Modal	Serverless GPU spins up in seconds; no image rebuild required
Frequent development changes	Modal	Avoids ECR image rebuild + push cycle
Infrastructure validation	AWS Batch	Exercises full Terraform stack (ECR, IAM, CloudWatch, EBS)

3.2.4 AWS Batch Backend

Location: `infra/backends/aws_backend.py`

Functionality. Submits a job to the Terraform-provisioned Batch queue, then polls CloudWatch logs every 30 seconds while the job is running. Logs are streamed to the terminal with a `[BATCH]` prefix.

Pre-condition. The Docker training image must be built and pushed to ECR (`tmoe-<env>-training`) prior to submission.

```
def run_aws_training(config, experiment_name, dry_run=False):
    s3_path = ensure_dataset_in_s3(config)
    if dry_run:
        return
    job_id = submit_batch_job(config, experiment_name)
    poll_batch_job(job_id, config)
```

Infrastructure Components Touched.

- AWS Batch Compute Environment (m5.xlarge, scales to zero)
- Job Queue and Job Definition (ECR image + resource limits)
- CloudWatch Log Group (14-day retention)
- Encrypted EBS volume (100GB gp3, deleted on termination)

At job completion, the container calls `upload_experiment_dir()`, syncing checkpoints, configuration, and metrics to S3.

3.2.5 Modal Backend

Location: `infra/backends/modal_backend.py`

Functionality. Provision a serverless GPU function on Modal, mount the source tree, and execute training without pre-built Docker images or persistent infrastructure.

Pre-condition.

- Modal CLI installed and authenticated

```
@app.function(
    gpu=config.modal_gpu,
    cpu=config.modal_cpu,
    timeout=config.modal_timeout,
    secrets=[modal.Secret.from_name("wandb-secret")],
)
def train_on_modal(experiment_config):
    s3_path = ensure_dataset_in_s3(config)
    execute_training_workflow(
        experiment_config,
        cache_dir="/cache"
    )
```

Key Properties.

- No idle infrastructure
- Billed per-second of GPU usage
- GPU type configurable in `config.yaml`
- No Terraform or image rebuild required

Upon completion, the same `upload_experiment_dir()` call syncs all outputs to S3, ensuring durability even after the ephemeral container exits.

3.2.6 Next Steps

Feature	Motivation
Automated pipeline triggers	Integrate EventBridge or GitHub Actions to automatically trigger ingestion + training on dataset updates
Formal data versioning	Integrate DVC or manifest-based version registry for reproducible dataset pinning
Preprocessing stage	Pre-tokenize, pack, and deduplicate text once and cache in S3 to reduce repeated compute
Multi-dataset blending	Support ingestion of multiple corpora and configurable dataset mixtures
Data quality monitoring	Add statistical profiling (nullity, length distribution, drift metrics) and persist quality reports alongside metadata

This unified training pipeline ensures compute flexibility, reproducibility, and infrastructure independence while maintaining a consistent experiment artifact structure across environments.

4 Part Four: Infrastructure as Code (IaC)

4.1 Terraform Infrastructure Architecture

Our Terraform infrastructure is built using a highly modular and decoupled design rather than relying on manual scripts or monolithic configuration files. This ensures that deployment code remains clean, maintainable, and reproducible across environments. The repository is organized by domain responsibility:

- **main.tf & backend.tf**
Acts as the centralized entry point to the infra deployment process.
 - Initializes AWS providers
 - Defines global default tags
- **vpc.tf & iam.tf**
Handles foundational networking and security layers. **vpc.tf** cleanly isolates our network routing, subnets, and security groups.
- **variables.tf & outputs.tf**
Serves as the “API” of the infrastructure. **variables.tf** parameterizes all resource configurations (like compute size and memory), and **outputs.tf** dynamically exports resource ARNs/URLs so our application code can consume them without hardcoding values.

4.2 Implementation of Multiple Environments

The infrastructure natively supports deploying to multiple environments (dev, staging, prod) from the exact same codebase.

This is achieved through the **environment** variable defined in **variables.tf**, which includes built-in validation rules to ensure only approved environment values are accepted.

All resources across the Terraform modules (like S3 buckets, ECR repos, and Batch queues) dynamically inject this environment variable into their naming conventions and tagging (e.g., **tmoe-dev-raw-data** vs **tmoe-prod-raw-data**). This allows us to spin up completely isolated production and testing environments simultaneously without resource collisions.

4.3 Infrastructure Completeness and Alignment with Parts 1–3

The Terraform code provisions the complete end-to-end infrastructure required for the entire ML lifecycle: data ingestion, model training, and experiment storage.

4.3.1 Data Storage and Ingestion (`s3.tf`, `sagemaker.tf`)

- Creates version-controlled S3 buckets for raw datasets and experiment artifacts
- Applies lifecycle policies (e.g., transitioning older objects to Glacier)
- Provisions SageMaker execution roles and permissions required for ingestion pipelines

This directly supports the ingestion pipeline described in Part Three.

4.3.2 Model Training and Compute (`batch.tf`, `ecr.tf`)

- Provisions an AWS Batch compute environment configured with GPU spot instances
- Automatically scales to zero when idle to minimize cost
- Defines job queues and job definitions for training workloads
- Creates ECR repositories to host Docker training images

The training containers built and pushed to ECR are dynamically pulled by Batch jobs at runtime.

4.3.3 Seamless Infrastructure–Application Handoff

The entire system is dynamically wired together through Terraform outputs and application configuration.

By treating infrastructure as code:

- Compute resources (vCPUs, memory, GPU type) are codified and reproducible
- Network routing and IAM permissions are version-controlled
- Application scripts consume dynamically exported infrastructure values
- The training environment is identical across runs and environments

This ensures full reproducibility of both infrastructure and experiments, aligning directly with the modular data ingestion and training pipelines described in Parts 1–3.

[Terraform Directory](#)

5 Part Five: Disaster Recovery Demonstration

Demo Video: [Watch Disaster Recovery Demo](#)